Informe de seguridad del frontend

Matriz de pruebas, aviso previo a la expiración y página Mi cuenta · PharmaSoft

C1. Resumen de ejecución

Se registraron 25 casos: los 16 de la actividad, cinco adicionales en grupos distintos y B-01 a B-04. Se observaron 23 resultados satisfactorios, cero fallos reproducibles y dos casos pendientes de evidencia manual. La batería de Angular terminó con 114 pruebas aprobadas en 21 archivos; la compilación de producción terminó correctamente.

Grupo	Casos	Pasa	Pendiente	Falla
Login	5	5	0	0
Sesión	5	4	1	0
Interceptor	4	3	1	0
Guards	3	3	0	0
Menú y acciones	3	3	0	0
Errores 401/403	1	1	0	0
Aviso de sesión	4	4	0	0
Total	25	23	2	0

S-04 queda pendiente porque no se midió la interacción entre dos pestañas. I-02 queda pendiente de captura manual: las pruebas automatizadas sí simulan la caída de conexión y verifican el mensaje y la sesión conservada, pero no se pudo documentar el panel Red del navegador. La sesión de Work informó que el navegador local estaba deshabilitado. Ambos estados se dejan pendientes y no se convierten en aprobados por inferencia.

El conteo Pasa indica que el comportamiento se verificó con la evidencia indicada en la matriz. No significa que cada fila tenga una captura: varias se respaldan con pruebas de Angular o respuestas sanitizadas del API. La consigna de una captura por caso queda incompleta para esas filas y debe cerrarse con el navegador habilitado.

Cambios implementados

AuthService calcula segundos restantes con signals, muestra el aviso cuando quedan 120 segundos o menos, permite ocultarlo hasta el próximo inicio de sesión y limpia temporizadores al salir. El JWT temporal del backend se firmó con 180 segundos para validar el umbral; la configuración persistente se dejó en 60 minutos. La ruta protegida /mi-cuenta consulta GET /api/v1/auth/me, presenta perfil, roles y permisos por módulo, y avisa cuando el servidor y la sesión difieren sin cambiar los permisos locales.

Hallazgo temporal de permisos

Se asignó a supervisor solo ALMACENERO para comprobar la discrepancia frente a la sesión iniciada. El API mostró el conjunto reducido; después se restauró ALMACENERO y VENDEDOR y se verificó la restauración. El cambio autorizado fue temporal y no alteró los datos de empleados. La lógica del aviso está cubierta por prueba automatizada; la captura visual de esa alerta queda pendiente por indisponibilidad del navegador integrado.

C2. Hallazgos y causas

No se reprodujeron fallos funcionales en los casos ejecutados. S-04 no se midió en dos pestañas; I-02 se verificó con una prueba automatizada de desconexión, pero falta su captura manual con el backend detenido. Además, varias filas aprobadas solo tienen evidencia de prueba automatizada o API, no la captura que pide la consigna. La causa de esta falta visual fue que Work informó que el navegador integrado estaba deshabilitado. No se propone una corrección de código sin un fallo reproducible. Para cerrar la evidencia, repetir S-04 e I-02, y tomar capturas de las filas marcadas con archivos de prueba, mostrando URL y paneles requeridos sin exponer el token.

1. Por qué el backend debe validar cada permiso

El menú, las directivas y los guards son controles de interfaz: el usuario puede modificar JavaScript, llamar una ruta escrita a mano o enviar una petición desde fuera de la SPA. Por eso el backend debe comprobar autorización en cada operación y recurso. En E-01 el usuario VENDEDOR intentó una baja sin permiso: el backend respondió 403 antes de buscar el producto; el frontend mostró el mensaje y conservó la sesión. Angular también advierte que nunca se debe depender solo de los guards del cliente [1].

2. sessionStorage, localStorage y cookie HttpOnly

sessionStorage persiste al recargar, pero se limita a una pestaña y termina con su sesión de navegación; localStorage comparte el valor entre pestañas y persiste más tiempo. Ambos son legibles por JavaScript del mismo origen y una vulnerabilidad XSS puede exponer el token. Una cookie HttpOnly impide que scripts lean su valor, y Secure/SameSite reducen riesgos de transporte y solicitudes cruzadas; aun así, el navegador la adjunta automáticamente, por lo que hay que cuidar CSRF. Para una botica desplegada elegiría una cookie HttpOnly, Secure y SameSite, idealmente con un BFF y protección CSRF. Para esta SPA académica se conserva sessionStorage porque lo requiere el diseño existente y reduce la persistencia frente a localStorage; no elimina el riesgo XSS [2][3].

Evidencia asociada

La matriz completa está en docs/matriz-seguridad-spa-s11.md. Las capturas con URL pertenecen a las pruebas aportadas; las pruebas automatizadas y las respuestas de API están identificadas como tales en la columna Evidencia. Los archivos de evidencia se mantienen separados para permitir su inspección.

C3. Preguntas de análisis

3. Por qué un 401 cierra sesión y un 403 no

Un 401 indica que la petición no tiene una autenticación aceptable: el token puede faltar, haber vencido o no ser válido. El interceptor elimina la sesión y devuelve al login, preservando returnUrl cuando procede. Un 403 indica que el usuario sí está autenticado, pero carece del permiso requerido; se conserva la sesión y se informa del rechazo. Si se cerrara sesión ante un 403, usuarios válidos perderían trabajo y volverían a autenticarse sin que sus permisos hubieran cambiado. Si se ignorara un 401, la interfaz mantendría un estado que el servidor ya no acepta y repetiría llamadas fallidas.

4. Redirección abierta y validación de returnUrl

Una redirección abierta aparece cuando la aplicación envía al usuario, tras completar una operación confiable, a un destino externo controlado por un atacante. El destino puede disfrazar una página de robo de credenciales como continuación del login. La aplicación debe aceptar solo rutas relativas internas bien formadas, rechazar esquemas, hosts externos, rutas que comienzan con // o variantes ambiguas, y usar /inicio como alternativa. En L-04 se probó un returnUrl HTTPS externo y el resultado observado fue /inicio, sin abandonar localhost.

Conclusión

El aviso de expiración y Mi cuenta quedan implementados y cubiertos por pruebas unitarias e integración de Angular; la API real confirmó los casos de 401, 403, perfil y cambio/restauración de roles. Las 114 pruebas y la compilación de producción pasaron. Antes de entregar, falta completar las dos verificaciones manuales pendientes con captura de navegador.

Referencias

[1] Angular, Control route access with guards. https://angular.dev/guide/routing/route-guards

[2] OWASP, Session Management Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

[3] MDN, Window: sessionStorage property. https://developer.mozilla.org/en-US/docs/Web/API/Window/sessionStorage